

Programação Orientada a Objetos

Introdução a Padrões de Projeto (Design Patterns)

Ana Claudia Bastos Loureiro Monção

anaclaudia@inf.ufg.br

Programação Orientada a Objetos

Definições



Padrão

"O padrão é uma descrição do problema e essência de sua solução, onde pode ser reutilizada em diversos casos. O padrão não é uma especificação detalhada, pode-se pensar como uma descrição de conhecimento e experiência acumulados." (SOMMERVILLE, 2003)

Programação Orientada a Objetos

Definições

- Em 1995, os profissionais: Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides, escreveram e lançaram o livro chamado “Design Patterns: Elements of Reusable Object-Oriented Software”, que mostra os detalhes dos 23 Design Patterns. Por essa realização, os profissionais foram batizados com o nome “Gangue dos Quatro” (Gang of Four ou GoF).
- Os objetivos dos padrões de projeto, são tornar os componentes reutilizáveis, que facilitam a padronização e que permitam agilidade para as soluções de problemas recorrentes no desenvolvimento do sistema.

Programação Orientada a Objetos

Motivação

- Projetar software OO reusável e de boa qualidade é difícil
- Aproveitar soluções de projetistas experientes
- Misturar soluções específicas com genéricas
- Impossível acertar da primeira vez

Programação Orientada a Objetos

Outros Tipos de Padrões:

- Padrões de Análise:
 - Descrevem grupos de conceitos que representam construções comuns na modelagem do domínio. Estes padrões podem ser aplicáveis em um domínio ou em muitos
- Padrões de Arquitetura:
 - Descrevem a estrutura e o relacionamento dos componentes de um sistema de software
- Padrões de Processo
 - Descrevem modelos de processo

Programação Orientada a Objetos

Padrões de Projeto GoF (Gang of Four)

- Esses padrões tem como objetivo, solucionar problemas comuns de softwares que tenham algum envolvimento com orientação a objetos.

São formados por três grupos:

- **Padrões de criação:** **Factory Method**, Abstract Factory, **Singleton**, Builder, Prototype.
- **Padrões estruturais:** Adapter, Bridge, **Composite**, Decorator, Facade, Flyweight, Proxy;
- **Padrões comportamentais:** Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, **Observer**, State, Strategy, Template, Method, Visitor.

Programação Orientada a Objetos

Características

- Algo comprovado - conhecimentos das melhores práticas
- Descoberto, não inventado
- Soluções Sucintas e de Fácil Aplicação
 - Capturam, de forma sucinta e facilmente aplicável, soluções de projeto de programas de computador que foram desenvolvidas e evoluíram com o passar do tempo
- Soluções Exaustivamente Refinadas
 - Resultados de um longo processo de projeto, reprojeto, teste e reflexão sobre o que torna um sistema mais flexível, reusável, modular e compreensível
- Soluções Compartilhadas
 - Construídas em grupo, através do uso de um vocabulário comum

Programação Orientada a Objetos

Documentação

- Nome:
 - Resume em uma ou duas palavras: o problema, as soluções e consequências do uso do padrão.
 - Deve ser facilmente lembrado, reflete o conteúdo do padrão.
- Problema:
 - Descreve quando aplicar o padrão. Explica o problema e seu contexto. Sintomas e condições.
- Solução:
 - Elementos que constituem o design, seus relacionamentos, responsabilidades e colaboradores.
- Consequências:
 - Resultados e compromissos decorrentes da aplicação do padrão.
 - Impactos sobre a flexibilidade, extensibilidade, portabilidade ou desempenho do sistema.

Programação Orientada a Objetos

Tipos de Padrões de Projeto

- Padrões GoF de **criação**
 - Exigem um tratamento de como os objetos são criados, para atender às diversas necessidades. No Java, os objetos são instanciados através de seus construtores, porém a utilização deles fica limitada quando:
 - O código que referencia a criação de um objeto precisa conhecer os construtores dele, isso aumenta o acoplamento das classes.
 - O objeto não pode ser criado parcialmente.
 - O construtor não consegue controlar o número de instâncias presentes na aplicação.



Programação Orientada a Objetos

Padrão de Criação **Factory Method**

- Nome:
 - **Factory Method**
- Problema:
 - Como adiar a decisão de que tipo de classe instanciar?
- Contexto:
 - Quando uma classe cria instâncias de outra classe, aumenta a dependência e o acoplamento entre elas, e é um grande problema para projetos. O ideal é tornar o acoplamento o menor possível.

Programação Orientada a Objetos

Padrão de Criação **Factory Method**

- Força:
 - Uso de classes abstratas e interfaces
 - Uso de uma fábrica de criação de objetos
 - Delegar a decisão de criação de objetos

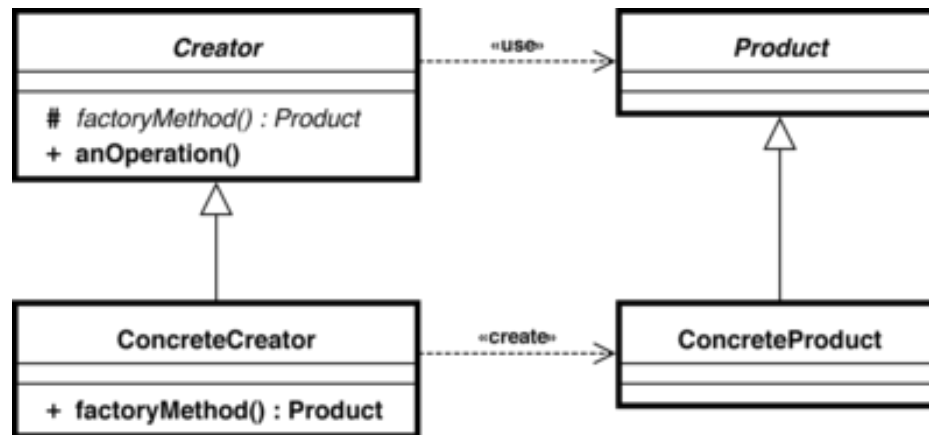
Programação Orientada a Objetos

Padrão de Criação **Factory Method**

- Solução:
 - Uma classe de criador abstrata que é a *Creator* que define um método fábrica abstrata que as subclasses implementam para criar um produto (*factoryMethod*) e pode possuir um ou mais métodos.
 - Cada *ConcreteCreator* implementará seu próprio método de criação.

Programação Orientada a Objetos

Padrão de Criação **Factory Method**



Programação Orientada a Objetos

Padrão de Criação **Factory Method**

- Vantagens:
 - encapsula-se a criação de objetos deixando as subclasses decidirem quais objetos criar – diminui o acoplamento
 - permite a uma classe decidir a instanciação para suas subclasses

Programação Orientada a Objetos

Padrão de Criação **Factory Method**

– Exemplo de Uso:

```
class abstract Fabrica {
    public abstract Pessoa criaPessoa(String nome,
                                      String sexo);
}
class FabricaPessoa extends Fabrica{
    public Pessoa criaPessoa(String nome,
                              String sexo) {
        if (sexo.equals("M"))
            return new Homem(nome);
        if (sexo.equals("F"))
            return new Mulher(nome);
    }
}

public class TesteApp {
    public static void main(String args[]) {
        FabricaPessoa factory = new FabricaPessoa();
        String nome = "Carlos";
        String sexo = "M";
        Pessoa pessoa = factory.criaPessoa(nome, sexo);
    }
}
```

```
public abstract class Pessoa {
    public String nome;
    public String sexo;
}

class Homem extends Pessoa {
    public Homem(String nome) {
        this.nome = nome;
        System.out.println("Olá Sr" + nome);
    }
}

class Mulher extends Pessoa {
    public Mulher(String nome) {
        this.nome = nome;
        System.out.println("Olá Sra" + nome);
    }
}
```

Programação Orientada a Objetos

Padrão de Criação **Singleton**

- Nome:
 - **Singleton**
- Problema:
 - Como pode ser construída uma classe que só pode ter uma única instância e que pode ser acessada globalmente dentro da aplicação?
- Contexto:
 - Em algumas aplicações, uma classe deve ter exatamente uma instância.
- Força:
 - Criar variáveis globais, usar variáveis de classe (estáticas)

Programação Orientada a Objetos

Padrão de Criação **Singleton**

- Solução:
 - Criar uma classe com um método de classe (ou estática) `getInstance()`, que:
 - Quando a classe é acessada pela primeira vez, a instância do objeto é criada e retornada para o cliente.
 - Nos acessos subsequentes ao método `getInstance()`, nenhuma instância adicional é criada, mas a identidade do objeto existente é retornada.

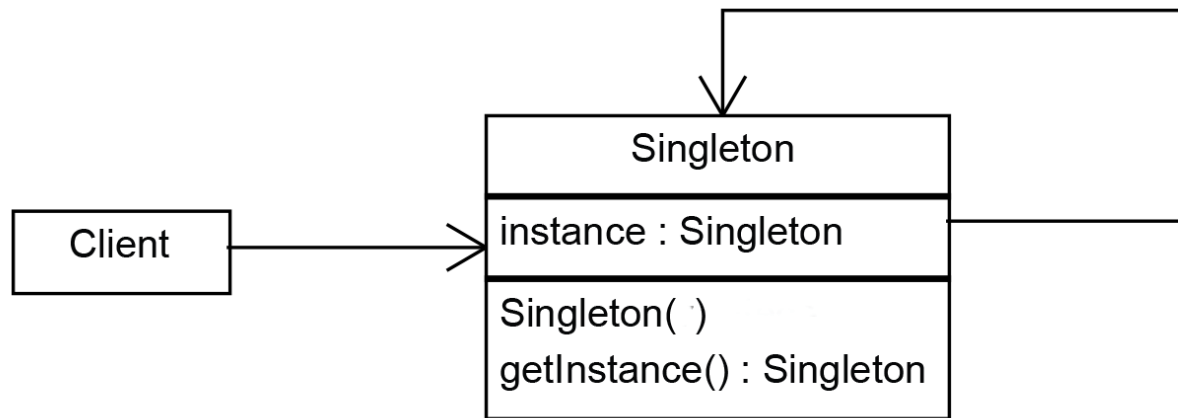
Programação Orientada a Objetos

Padrão de Criação **Singleton**

- Alguns aspectos que devem ser observados ao criar esse padrão:
 - O construtor da classe fica como privado (private), sendo que não pode ser instanciada para fora da própria classe.
 - A classe é final, pois não permite a criação de subclasses da própria classe.
 - O acesso é permitido através do método que retorna a instância única da classe, ou faz a criação de uma, caso não tenha sido criada.

Programação Orientada a Objetos

Padrão de Criação **Singleton**



Programação Orientada a Objetos

Padrão de Criação **Singleton**

- Implementação:

```
class Singleton {  
    private static Singleton instance;  
  
    private Singleton() {  
        // Construtor privado: impede "new Singleton()"  
    }  
  
    public static Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

Programação Orientada a Objetos

Padrão de Criação **Singleton**

- Vantagens:
 - Oferece acesso controlado à única instância do objeto, pois a classe **Singleton** encapsula a instância.
 - Uma variação do padrão pode ser usada para criar um número específico de instâncias, se for requerido.

Programação Orientada a Objetos

Padrão de Criação Singleton

- Exemplo de Uso:
 - Uma aplicação bancária onde cada conta tem um vínculo com um banco. O banco deve ser único para todas as contas.

```
public final class Banco {  
    public static Banco instancia;  
    int codigo;  
    String nome;  
    private Banco() {  
        codigo = 001;  
        nome = "Banco do Brasil";  
    }  
    public static Banco getInstancia() {  
        if (instancia == null) {  
            instancia = new Banco();  
        }  
        return instancia;  
    }  
}
```

```
public class Conta {  
  
    Banco banco;  
    int codigo;  
    String nome;  
  
    Conta(int codigo, String nome) {  
        banco = Banco.getInstancia();  
        this.codigo = codigo;  
        this.nome = nome;  
    }  
}
```

Programação Orientada a Objetos

Tipos de Padrões de Projeto

- Padrões GoF **Estruturais**
 - Esse padrão descrevem os seguintes aspectos: elaboração, associação e a organização entre objetos e classes/interfaces. Permitem combinar objetos em estruturas mais complexas, ou descrever como as classes são herdadas ou compostas a partir de outras.



Programação Orientada a Objetos

Padrão **Estrutural** Composite

- Nome:
 - **Composite**
- Problema:
 - Se existe um requisito para representar hierarquias todo parte, então ambos objetos (todo e parte) oferecem a mesma interface para objetos cliente.
- Contexto:
 - Numa aplicação tanto o objeto que contém quanto o que é componente devem oferecer o mesmo comportamento.
 - Objetos cliente devem ser capazes de tratar objetos compostos ou componentes do mesmo jeito.

Programação Orientada a Objetos

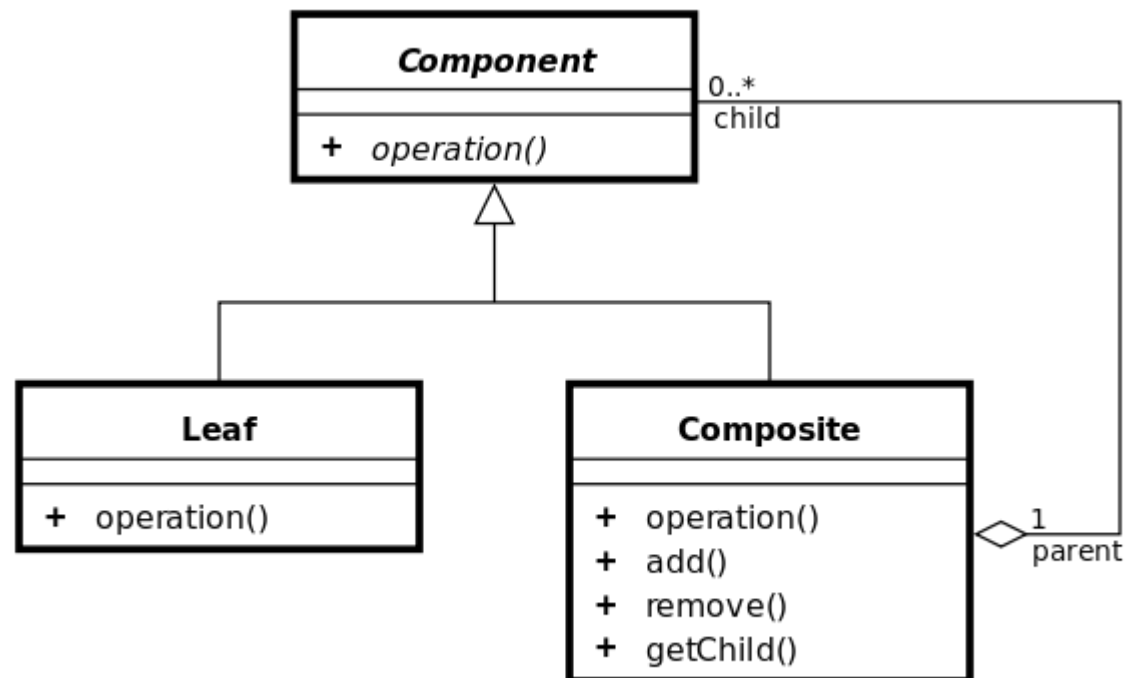
Padrão Estrutural Composite

- Forças:
 - O requisito que os objetos, tanto composto como componente, ofereçam a mesma interface, sugere que eles pertençam à mesma hierarquia de herança.
 - Isto permite que operações sejam herdadas e redefinidas com a mesma assinatura (polimorfismo).
 - A necessidade de representar hierarquias toda parte indica a necessidade de uma estrutura de agregação

Programação Orientada a Objetos

Padrão Estrutural **Composite**

- Solução: Combinar hierarquias de herança e agregação



Programação Orientada a Objetos

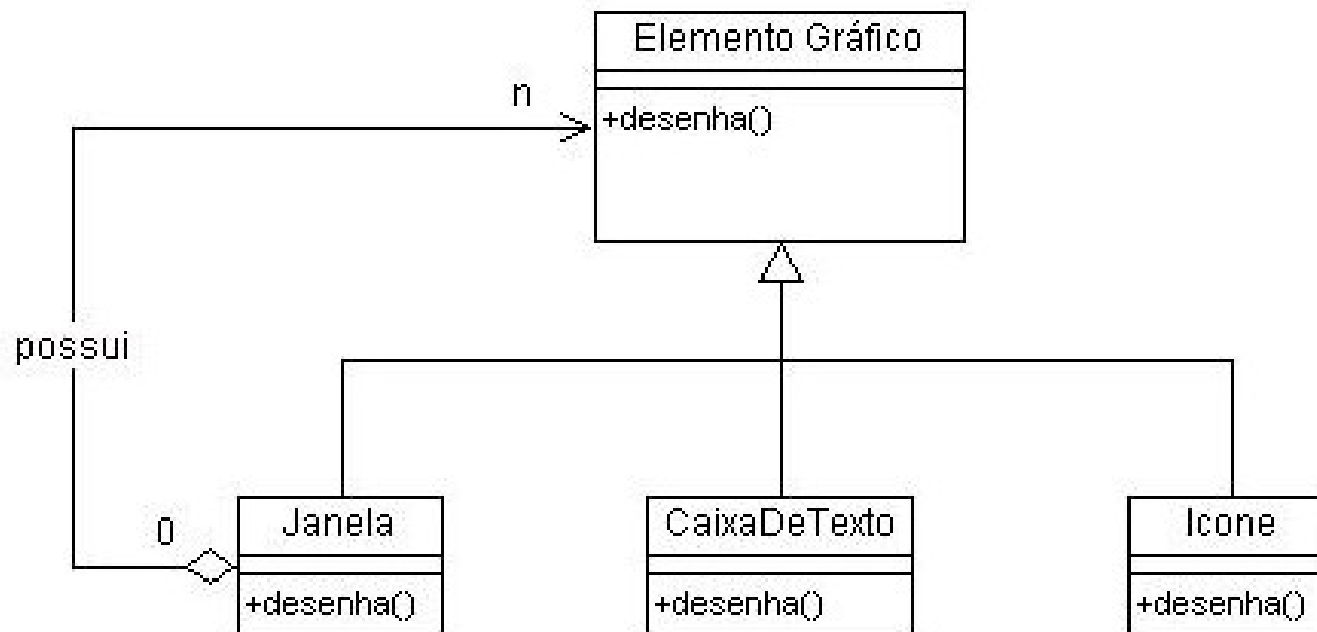
Padrão Estrutural **Composite**

- Solução:
 - Ambas subclasses, *Leaf* e *Composite*, têm uma operação redefinida usando polimorfismo *anOperation()*.
 - Na classe *Composite* esta operação redefinida invoca a operação relevante a partir de seus componentes usando um *loop*.
 - A subclasse *Composite* também tem operações adicionais para gerenciar a hierarquia de agregação, então os componentes podem ser adicionados ou removidos.

Programação Orientada a Objetos

Padrão Estrutural Composite

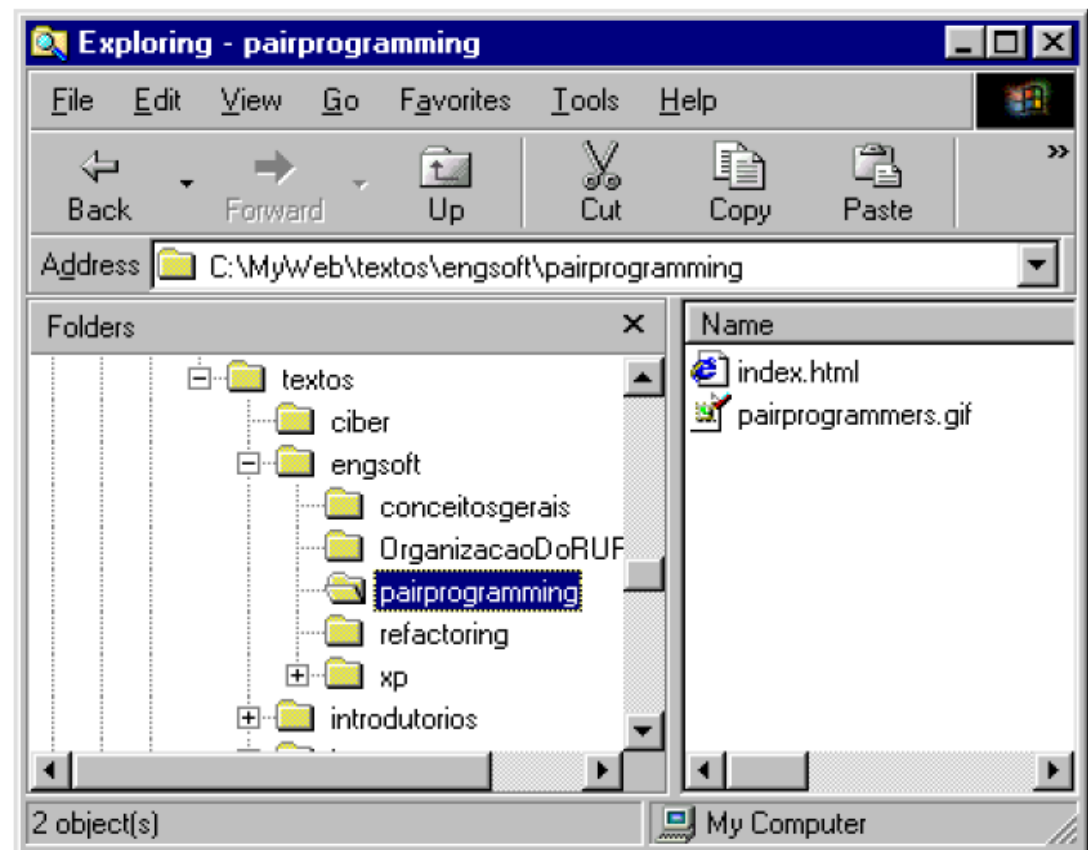
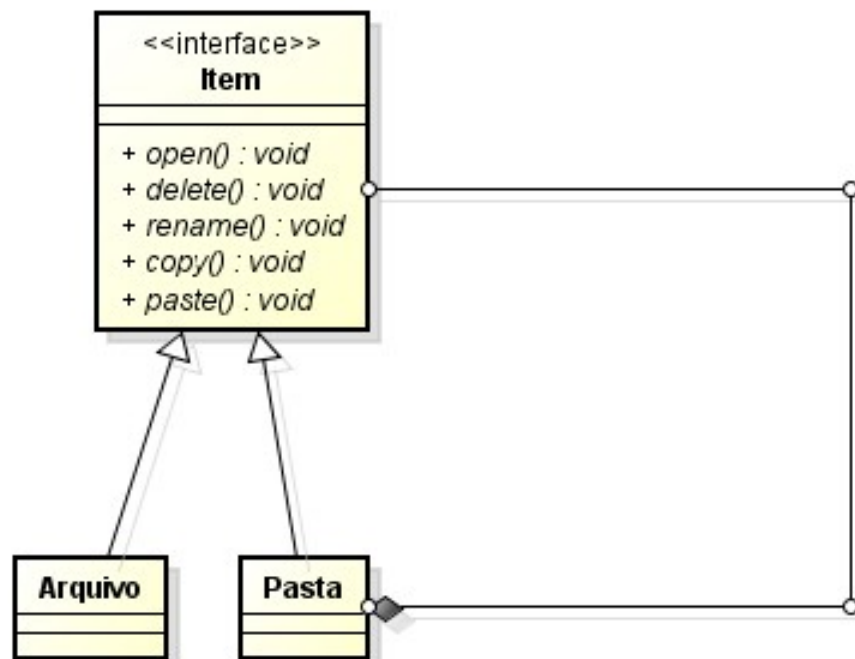
- Exemplo : Telas



Programação Orientada a Objetos

Padrão Estrutural Composite

- Exemplo : Sistema de Arquivos



Programação Orientada a Objetos

Tipos de Padrões de Projeto

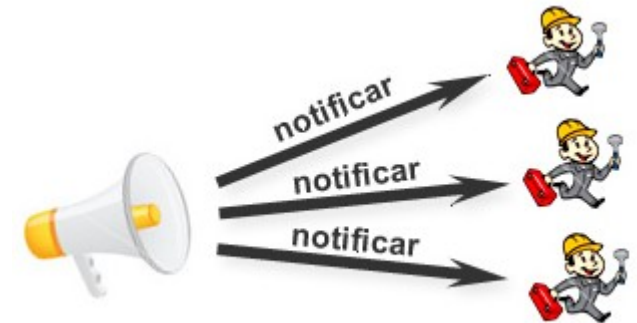
- Padrões GoF Comportamentais
 - Esses padrões, mostram o processo de como os objetos ou classes se comunicam. Em geral, buscam um **baixo acoplamento** entre os objetos, apesar da comunicação que existe entre eles.



Programação Orientada a Objetos

Padrão Comportamental **Observer**

- Nome:
 - Observer
- Problema:
 - Os dados do componente sujeito modificam-se constantemente e precisam ser atualizados nos outros componentes. O número de componentes pode variar.
- Contexto:
 - Situações nas quais vários componentes dependem de dados que são modificados em outro componente (sujeito).



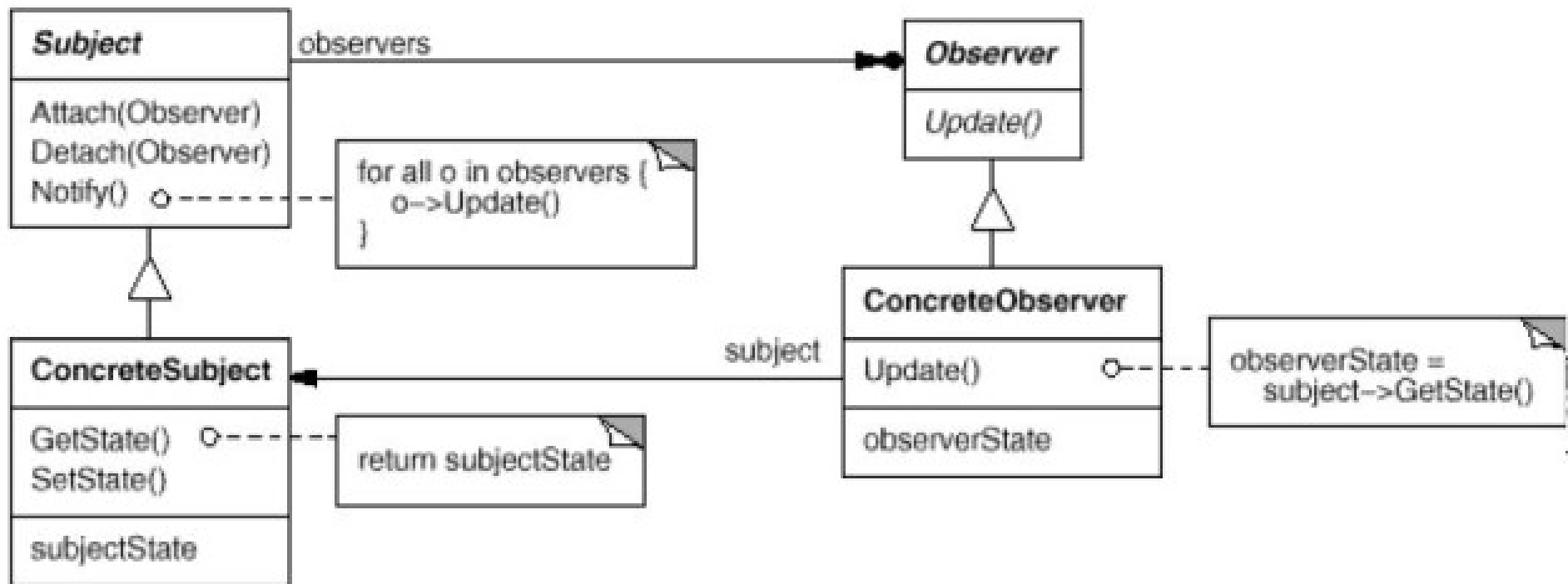
Programação Orientada a Objetos

Padrão Comportamental **Observer**

- Solução:
 - Utilizar um mecanismo de registro que permite ao componente sujeito notificar aos interessados sobre mudanças.
 - Exemplos: atualizações de software, facebook, pacote swing (listeners), etc.

Programação Orientada a Objetos

Padrão Comportamental **Observer**



Programação Orientada a Objetos

Padrão Comportamental Observer

Interface pública Observer: As classes que implementarem essa interface, que por sua vez conterá um método de atualização (update), deverão ser notificadas quando da atualização ou mudança do estado do objeto observado.

Classe Observable: Define a classe que será observada. Essa classe geralmente guarda como atributo uma lista dos observers associados à mesma, para que os mesmos possam ser notificados quando a mudança acontecer na classe.

java.util.Observable

- + Observable()
- + addObserver(o: Observer)
- + countObservers(): int
- + deleteObserver(o: Observer)
- + deleteObservers()
- + hasChanged(): boolean
- + notifyObservers(arg: Object)
- + notifyObservers()

«interface»

java.util.Observer

- + update(o: Observable, arg: Object)

Programação Orientada a Objetos

Padrão Comportamental Observer

```
public class Dieta extends Observable {
    private List observers = new ArrayList();
    private double peso;

    public Dieta(double peso) {
        this.peso = peso;
    }

    public void setPeso(double peso) {
        this.peso = peso;
        // Quando a alteração do peso ocorre notifica os observadores.
        this.notifyObservers();
    }

    public void registerObserver(Observer observer) {
        observers.add(observer);
    }

    public void removeObserver(Observer observer) {
        observers.remove(observer);
    }

    public void notifyObservers() {
        // Chama o método de atualização dos observers disponíveis.
        for (Observer ob : observers) {
            System.out.println("Notificando observers!");
            ob.update(this.peso);
        }
    }
}
```

```
// Classe observadora "Balança"
class BalancaDieta implements Observer {

    public void update(double novoPeso) {
        if (peso >= 100) {
            System.out.println
                ("Você ultrapassou o peso limite no sistema!");
        }
    }
}

public class Teste {

    public static void main(String args[]) {
        BalancaDieta balanca = new BalancaDieta();
        Dieta dieta = new Dieta();
        dieta.registerObserver(balanca);
        dieta.setPeso(150);
    }
}
```

Programação Orientada a Objetos

Questões Relacionadas ao uso de Padrões:

- Existe algum padrão que trata um problema similar?
- O contexto do padrão é consistente com o problema real?
- O padrão tem uma solução alternativa que pode ser mais aceitável?
- Existe uma solução mais simples?
- Existem algumas restrições que sejam impostas pelo ambiente de software que está sendo usado?